

## Programmierung 2 - Sommersemester 2026

Prof. Dr. Markus Esch

### Übung Nr. 5 Abgabe KW 22

#### 1. Aufgabe (Codeopolis)

Erweitern Sie die Klasse `LinkedList` aus der vorherigen Übung um eine Methode `sort`, die die Elemente der Liste mit Hilfe des Bubblesort-Algorithmus sortiert ( Wenn Sie möchten, können Sie auch einen komplexeren Sortieralgorithmus implementieren).

Um die Elemente der Liste sortieren zu können, müssen Sie sicherstellen, dass sie vergleichbar sind. Verwenden Sie dazu das Interface `java.lang.Comparable<T>`<sup>1</sup>, welches wir in der Vorlesung kennengelernt haben. Stellen Sie sicher, dass die Elemente in der Liste dieses Interface implementieren.

Um die Klassen `Silo` und `Harvest` weiterhin als Elemente in der Liste verwenden zu können, müssen diese das Interface implementieren. Vergleichen Sie `Silo`-Objekte anhand des Füllstands und `Harvest`-Objekte anhand des Erntejahres.

Passen Sie anschließend die Methode `Depot.toString()` so an, dass die Silos des Depots sortiert nach dem Füllstand ausgegeben werden.

#### 2. Aufgabe

- (a) Implementieren Sie eine Methode `applyAndPrint`, die zwei natürliche Zahlen  $i$  und  $j$  sowie eine Funktion  $f : \mathbb{N} \rightarrow \mathbb{N}$  als Argumente entgegennimmt. Die Methode soll die übergebene Funktion auf alle Zahlen zwischen  $i$  und  $j$  anwenden und das Ergebnis ausgeben. Verwenden Sie zur Übergabe der Funktion folgendes funktionales Interface oder ein Interface aus dem Package `java.util.function`:

```
public interface MyFunction{  
    public int apply(int i);  
}
```

- (b) Rufen Sie die `applyAndPrint`-Methode mit den im Folgenden aufgeführten Funktionen auf. Implementieren Sie die Funktionen jeweils als Lambda-Ausdruck und als anonyme Klasse. Implementieren Sie die Funktion *ii* zusätzlich als Top-Level-Klasse und als echte innere Klasse. Vergleichen Sie den Aufwand für die Implementierung und diskutieren Sie im Review Vor- und Nachteile.

i  $f(x) = x^2$

ii  $f(x) = x!$

---

<sup>1</sup><https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Comparable.html>

- iii  $f(x) = x^{(x+1)}$
  - iv  $f(x) = fib(x)$  (Fibonacci-Folge)
- (c) Implementieren Sie ein funktionales Interface, welches das in Aufgabe (a) verwendete Interface erweitert. Dieses Interface soll die beiden folgenden zusätzlichen Methoden mit Default-Implementierung besitzen:
- i `conditionateInput`: Diese Methode nimmt ein Prädikat als Argument entgegen und gibt ein Objekt vom Typ des Interfaces in Form eines Lambda-Ausdrucks zurück. Der Lambda-Ausdruck prüft für einen gegebenen Integer-Wert, ob dieser das Prädikat erfüllt. Ist das Prädikat erfüllt, wird das Ergebnis der `apply`-Methode des Interfaces zurückgegeben, ansonsten wird 0 zurückgegeben.
  - ii `conditionateOutput`: Diese Methode nimmt ebenfalls ein Prädikat als Argument entgegen und gibt ein Objekt vom Typ des Interfaces in Form eines Lambda-Ausdrucks zurück. Der Lambda-Ausdruck prüft, ob das Ergebnis der `apply`-Methode das Prädikat erfüllt. Ist das Prädikat erfüllt, wird das Ergebnis zurückgegeben, ansonsten 0.
- (d) Implementieren Sie ein Prädikat `odd` und ein Prädikat `even`. Die Prädikate sollen prüfen, ob ein gegebener Integer-Wert gerade bzw. ungerade ist. Implementieren Sie das `odd`-Prädikat als anonyme Klasse und das `even`-Prädikat als Lambda-Ausdruck.
- (e) Rufen Sie die Methode `applyAndPrint` mit einem Lambda-Ausdruck auf, welcher für gerade Zahlen die Quadratzahl ausgibt. Nutzen Sie dabei das `even`-Prädikat sowie das Interface aus Aufgabe (c).
- (f) Rufen Sie die Methode `applyAndPrint` mit einem Lambda-Ausdruck auf, welcher prüft, ob die Fibonacci-Zahl einer gegebenen Zahl ungerade ist. Ist dies der Fall, wird die Fibonacci-Zahl ausgegeben, ansonsten 0. Nutzen Sie dabei das `odd`-Prädikat sowie das Interface aus Aufgabe (c).

### 3. Aufgabe (Codeopolis)

In dieser Aufgabe soll die Klasse `LinkedList` erweitert werden, um den Zugriff auf die in der Liste gespeicherten Elemente zu verbessern. Fügen Sie der Klasse die folgenden Methoden hinzu:

- **filter**: Die Methode nimmt ein `Predicate` entgegen und gibt eine `LinkedList` mit den Elementen zurück, für die dem `Predicate` genügen.
- **forEach**: Die Methode erhält als Argument einen `Consumer` und wendet diesen auf alle Elemente der Liste an.
- **removeIf**: Die Methode entfernt alle Elemente aus der Liste, die einem der Methode übergebenen `Predicate` entsprechen.
- **addIf**: Die Methode nimmt ein neues Element und ein `Predicate` entgegen. Das Element wird der Liste hinzugefügt, wenn es dem Kriterium des `Predicate`s entspricht.
- **sort**: Die Methode erhält als Argument ein Objekt vom Typ `java.util.Comparator`<sup>2</sup> und sortiert die Elemente der Liste anhand dieses `Comparators`.

<sup>2</sup><https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Comparator.html>

## Lernziele:

Nach der Bearbeitung des Übungsblattes sollten Sie ...

- ... Generics und insbesondere Wildcards und Bounded Typ Parameter erklären und nutzen können.
- ... in der Lage sein ein funktionale Interfaces zu nutzen.
- ... die Syntax vom Lambda-Ausdrücken erklären und anwenden können.
- ... in der Lage sein einfache mathematische Funktionen als Lambda-Ausdruck zu implementieren.
- ... die Unterschiede der Implementierung mit Lambda-Ausdrücken und inneren Klassen erklären können.
- ... default-Methoden von funktionalen Interface implementieren und nutzen können.
- ... das für eine Aufgabe passende funktionale Interface auswählen und nutzen können.
- ... in der Lage sein funktionale Interfaces für die Implementierung von Operationen auf einer Datenstruktur zu nutzen.

## Zusatzaufgabe

Die folgenden Aufgaben sind Zusatzaufgaben und nicht Teil des Peer Reviews! Sie können die Aufgaben freiwillig bearbeiten, um die Inhalte weiter zu üben. Selbstverständlich können Sie auch Unterstützung und Beratung zu den Zusatzaufgaben erhalten.

### 1. Aufgabe (Codeopolis)

Implementieren Sie JUnit-Tests für die neuen Funktionen der `LinkedList`.

### 2. Aufgabe

(a) Implementieren Sie eine Methode `iterate`. Diese nimmt folgende Werte als Parameter entgegen:

i  $x_0$  = Startwert (double)

ii  $n$  = Iterationszahl (int)

iii  $f : \mathbb{R} \rightarrow \mathbb{R}$  = Funktion

Die Funktion  $f$  soll wiederholt ( $n$  mal) wie folgt angewendet werden:  $x_{i+1} = f(x_i)$ . Das Ergebnis der Berechnung wird als Double-Wert zurückgegeben. Definieren Sie ein passendes funktionales Interface zur Übergabe der Funktion  $f$  oder nutzen Sie ein Interface aus dem Package `java.util.function`.

(b) Rufen Sie die Methode `iterate` mit folgenden Funktionen auf:

i  $f(x) = 2x$

ii  $f(x) = 0.5x$

iii  $f(x) = ax(x - 1)$ , für ein gegebenes  $a \in (0, 1)$

Implementieren Sie die Funktionen als Lambda-Ausdrücke.